

Labo 19 – Verslag JavaScript deel 7

opleidingsonderdeel **Web Development I**

Bachelor in de **toegepaste informatica**

campus **Kortrijk**

Tibo Dewanckele

docent: **Pim Debaere**

academiejaar **2025–2026**



Inhoud

Demo's.....	2
Sprites	2
Timers.....	3
Hit an object	4
Matching game.....	8

Demo's

Sprites

Klik op het speelveld om de smiley te verplaatsen.



```
let speelveld=document.getElementById("speelveld");
speelveld.style.width=window.innerWidth+"px";
speelveld.style.height=window.innerHeight+"px";
```

Om het speelveld aan te passen aan de grootte van het scherm waarop je aan het kijken of als je de grootte verandert.

Via **window.innerWidth** en **window.innerHeight** kan je deze krijgen.

We gebruiken voor een HTML element **clientWidth** en **clientHeight** voor breedte en hoogte.

Je moet hierachter nog **“px”** zetten want dit is anders nog tekort.

```

let sprite=document.getElementsByClassName("sprite")[0];
let speelveld=document.getElementById("speelveld");
let maxLeft=speelveld.clientWidth - sprite.offsetWidth;
let maxHeight=speelveld.clientHeight - sprite.offsetHeight;

let left=Math.floor(Math.random()*maxLeft);
let top=Math.floor(Math.random()*maxHeight);
sprite.style.left=left+"px";
sprite.style.top=top+"px";

```

Hier halen we de sprite op en dan berekenen we aan de hand van het speelveld en de grootte van de sprite wat het maximum aan de linker- en bovenkant.

Math.random geeft een getal tussen 0 en 1 waardoor dat je het maal het maximum waarde moet doen. Dan via **Math.floor** rond je het af naar beneden omdat we geen kommagetal willen. We kunnen ook afronden via **Math.ceil** (naar boven) of **Math.round** (wiskundig)

Timers

tick tick tick tick

Stop

```

let timerId=0;

const initialize = () => {
  let btnStop=document.getElementById("btnStop");
  btnStop.addEventListener("click", stopTimer);
  timerId=setInterval(timerTick, 1000);
}

const timerTick = () => {
  let output=document.getElementById("output");
  output.innerHTML+=" tick";
}

const stopTimer = () => {
  clearInterval(timerId);
}

window.addEventListener("load", initialize);

```

Als we op de knop drukken willen we dat de timer stopt.

We zetten een **setInterval** om de 1000ms waar dat we timerTick uitvoeren.

Elke setInterval heeft een eigen **id** dat gebruikt worden om het uitvoeren van timerTick om de seconde te stoppen.

Dit gebeurt via **clearInterval**.

```
const initialize = () => {
  let btnOpnieuw=document.getElementById("btnOpnieuw");
  btnOpnieuw.addEventListener("click", herstartTimer);
  setTimeout(timerTick, 1000);
}

const timerTick = () => {
  let output=document.getElementById("output");
  output.innerHTML+=" tick";
}

const herstartTimer = () => {
  setTimeout(timerTick, 1000);
}

window.addEventListener("load", initialize);
```

Hier willen we dat bij het drukken van de knop na een seconde tick wordt bijgevoegd. Als we op de knop drukken dan wordt herstartTimer opgeroepen waarin **setTimeout** wordt gebruikt om een seconde te wachten en dan timerTick uit te voeren.

Je kan een timeout ook gebruiken door in een methode de methode zelf aan te roepen in de **setTimeout**.

Om deze dan terug te stoppen moet je de **id** toekennen wanneer je **setTimeout**.

Via **clearTimeout** stop je de timeout dan.

Hit an object

Score: 2



Startsituatie is een startknop met daaronder een foto van een bom (0.png).

De afbeelding heeft een id Target waarbij dat we in css position absolute zetten zodat we deze makkelijk kunnen verplaatsen.

```
let global = {
  IMAGE_COUNT: 5,
  IMAGE_SIZE: 48,
  IMAGE_PATH_PREFIX: "images/",
  IMAGE_PATH_SUFFIX: ".png",
  MOVE_DELAY: 3000,
  score: 0,
  timeoutId: 0
};
```

We roepen bovenaan in javascript de globale variabelen aan die we in de code via **global**.variabeleNaam kunnen aanroepen.

Om te starten wachten we totdat er op de startknop wordt gedrukt.

```
const setup = () => {
  const start = document.querySelector("#start");
  start.addEventListener("click", startSpel)
};
```

Als er op is gedrukt dan starten we het spel.

We resetten eerst alles naar de beginsituatie als we het eerder al hebben gespeeld.

Dan halen we de afbeelding op en dan gaan we een methode oproepen die controleert of hij een alert moet geven of de sprite moet verplaatsen.

Dan gaan we ook automatische acties uitvoeren als we niets aan het doen zijn.

We doen deze actie alvast 1 keer zodat we niet moeten wachten totdat hij 1 keer verplaatst. Dan maken we ook nog een score naast de startknop aan.

```
const startSpel = () => {
  reset();
  const sprite = document.querySelector("#target");
  sprite.addEventListener("click", alertOfVerplaats);

  automatischeActie();
  global.timeoutId = setInterval(automatischeActie, global.MOVE_DELAY);
  maakScoreDisplayAan();
}
```

Om deze score naast de knop te krijgen halen we de div aan hiervoor en dan creëren we een span met hierin de score als textContent en dan voegen we het toe aan de div.

```
const maakScoreDisplayAan = () => {
  let hoofdDiv = document.querySelectorAll("div")[0];
  let scoreSpan = document.createElement("span");
  scoreSpan.id = "scoreSpan";
  scoreSpan.textContent = "Score: " + global.score;
  hoofdDiv.appendChild(scoreSpan);
}
```

We willen deze display ook updaten iedere keer er een punt wordt gescoord.

```
const updateScoreDisplay = () => {  
  global.score++;  
  let scoreSpan = document.querySelector("#scoreSpan");  
  scoreSpan.textContent = "Score: " + global.score;  
}
```

De reset gebeurt door globale variabelen timeoutId en score terug op 0 te zetten en de display van de score terug te verwijderen.

```
const reset = () => {  
  global.timeoutId = 0;  
  global.score = 0;  
  
  let scoreSpan = document.querySelector("#scoreSpan");  
  if(scoreSpan) {  
    scoreSpan.remove();  
  }  
}
```

De automatische actie die eigenlijk om de seconde moet uitgevoerd worden bestaat uit 2 methodes en een bericht op de console. We gaan de sprite verplaatsen en veranderen.

```
const automatischeActie = () => {  
  verplaatsSprite();  
  veranderSprite();  
  console.log(global.score);  
}
```

Om de sprite te verplaatsen halen we de sprite en het speelveld op. We berekenen het veld waarin de sprite kan geplaatst kan worden. Dan laten we via random bepalen waar de sprite landt en kennen dit toe aan de sprite.

```
const verplaatsSprite = () => {  
  const sprite = document.querySelector("#target");  
  const playField = document.getElementById("playField");  
  
  const maxX = playField.clientWidth - global.IMAGE_SIZE;  
  const maxY = playField.clientHeight - global.IMAGE_SIZE;  
  
  const randomX = Math.random() * maxX;  
  const randomY = Math.random() * maxY;  
  
  sprite.style.left = randomX + "px";  
  sprite.style.top = randomY + "px";  
}
```

Om de sprite te veranderen laten we een random getal kiezen uit het aantal afbeeldingen en via de naamgeving van de afbeelding "0.png", "1.png", ... om dan het aan de src via **setAttribute** toe te kennen. We gebruiken de globale variabelen om de constanten hier te gebruiken.

```
const veranderSprite = () => {
  const sprite = document.querySelector("#target");

  const randomIndex = Math.floor(Math.random() *
    global.IMAGE_COUNT);
  sprite.setAttribute("src", global.IMAGE_PATH_PREFIX +
    randomIndex + global.IMAGE_PATH_SUFFIX);
}
```

Als het 0.png (de bom) is dan geven we een alert aan de gebruiker en stoppen we het spel. In alle andere gevallen gaan we de sprite verplaatsen, veranderen en de score updaten.

```
const alertOfVerplaats = () => {
  const sprite = document.querySelector("#target");
  if(sprite.getAttribute("src")===global.IMAGE_PATH_PREFIX + 0 +
    global.IMAGE_PATH_SUFFIX) {
    alert("Je hebt geklikt op de bom");
    stopSpel();
  }else {
    verplaatsSprite();
    veranderSprite();
    updateScoreDisplay();
  }
}
```

Om het spel te stoppen gebruiken we de **timeoutId** om **clearInterval** te gebruiken. Zo kunnen we die actie stoppen. We verwijderen ook nog de eventlistener van de sprite om te voorkomen dat er nog iets wordt gedaan na het stoppen van het spel.

```
const stopSpel = () => {
  clearInterval(global.timeoutId);

  const sprite = document.querySelector("#target");
  sprite.removeEventListener("click", alertOfVerplaats);
}
```

Matching game

Dit zijn de globale variabelen voor deze opdracht

```
let global = {
  AANTAL_HORIZONTAAL: 4,
  AANTAL_VERTICAAL: 3,
  AANTAL_KAARTEN: 6,
  isBusy: false
}
```

We starten met een spelbord te genereren.

```
const setup = () => {
  genereerSpelbord();
}
```

We genereren een spelbord waarbij dat we een **id** spelbord meegeven die in de css ook al eigenschappen krijgt waarbij de **display: grid**.

Omdat het aantal rijen en kolommen in een variabele zit wordt dit gedefinieerd in javascript. Omdat dit moet toegevoegd worden in HTML voor de script tag gebruiken we **prepend**. Hierachter genereren we de kaarten op het spelbord.

```
const genereerSpelbord = () => {
  let body = document.querySelector("body");

  let spelbord = document.createElement("div");
  spelbord.setAttribute("id", "spelbord");
  spelbord.style.gridTemplateColumns = "repeat(" + global.AANTAL_HORIZONTAAL + ", 1fr)";
  spelbord.style.gridTemplateRows = "repeat(" + global.AANTAL_VERTICAAL + ", 1fr)";

  body.prepend(spelbord);
  genereerKaartenOpSpelBord();
}
```


We vullen eerst de kaarten op zodat elke afbeelding er 2 keer in zit.

We gaan dan de grid opvullen met div tags waarin elk een img tag zit.

Ik voeg ook nog de class kaart in voor de css en dan voegen we dit toe aan het spelbord.

Zoals gezegd maken we nog de img tag met als attributes src waarbij de foto momenteel altijd achterkant is van de kaart en ook nog welke afbeelding de voorkant van de kaart is.

We kijken dan wanneer de kaart is aangeklikt, dat deze wordt omgedraaid.

```
const genereerKaartenOpSpelBord= () => {
  let kaarten = vulKaarten();
  let spelbord = document.querySelector("#spelbord");
  for(let i = 0; i < global.AANTAL_HORIZONTAAL; i++) {
    for (let j=0; j < global.AANTAL_VERTICAAL; j++) {
      let div = document.createElement("div");
      div.classList.add("kaart");
      spelbord.append(div);

      let img = document.createElement("img");
      img.setAttribute("src","images/achterkant.png");
      img.setAttribute("kaart", kaarten[i*global.AANTAL_VERTICAAL+j]);
      div.append(img);

      img.addEventListener("click", draaiKaartOm);
    }
  }
}
```

Om de kaarten te vullen halen we alle afbeeldingen op en steken we de afbeeldingen er elk 2 keer in. Om deze random door elkaar te steken gebruiken we **sort** en om het makkelijk **sorteer algoritme** te gebruiken, maken we gebruik van **Math.random() - 0.5**

```
const vulKaarten = () => {
  let images = geefAfbeeldingen();
  let kaarten = [];

  images.forEach(img => {
    kaarten.push(img);
    kaarten.push(img);
  });

  kaarten.sort(() => Math.random() - 0.5);
  return kaarten;
};
```

Bij het omdraaien van een kaart controleren we of dat we nog een kaart kunnen omdraaien via `isBusy`. We spelen een geluid af via `new Audio` en het pad van het geluid. Via `play` kunnen we het afspelen.

We gebruiken **`event.target`** om de afbeelding waarop is geklikt op te halen. Hiervan halen we de kaart attribute en deze gebruiken we dan in de nieuwe `src` voor de voorkant. Als er dan 2 kaarten zijn gedraaid dan zeggen we dat er niet nog kaarten kunnen gedraaid worden via `isBusy`.

Dan is er controle of deze matchen, als ze matchen zetten we hun border op groen en via een timeout verwijderen we dit paar na een seconde.

Als het geen match is dan wordt de border rood en draaien we deze terug.

```
const draaiKaartOm = (event) => {
  if (global.isBusy) return;
  let audio = new Audio('sounds/kaart_omdraaien.mp3');
  audio.play();
  let img = event.target;
  let kaart = img.getAttribute("kaart");
  img.setAttribute("src", "images/" + kaart);
  if(hoeveelKaartenGedraaid()===2){
    global.isBusy = true;
    if(zijnGedraaideKaartenMatch()){
      let images = document.querySelectorAll("img");
      images.forEach(img => {
        if (isKaartGedraaid(img)) {
          img.parentElement.style.border = "3px solid green";
        }
      });
      setTimeout(verwijderPaar,1000);
    }else{
      let images = document.querySelectorAll("img");
      images.forEach(img => {
        if (isKaartGedraaid(img)) {
          img.parentElement.style.border = "3px solid red";
        }
      });
      setTimeout(draaiKaartenTerug,1000);
    }
  }
}
```

Om te tellen hoeveel kaarten zijn gedraaid tellen we hoeveel kaarten als src niet de achterkant hebben. Als we ze doen verdwijnen dan zetten we de src op de achterkant zodat we hier geen probleem hebben. Dit wordt gecontroleerd in isKaartGedraaid.

```
const hoeveelKaartenGedraaid = () => {
  let images = document.querySelectorAll("img");
  let aantalGedraaid = 0;
  images.forEach(img => {
    if(isKaartGedraaid(img)){
      aantalGedraaid++;
    }
  })
  return aantalGedraaid;
}
```

Om te controleren of ze matches halen we alle src's op die niet de achterkant zijn en omdat er maar 2 zijn kunnen we deze vergelijken met elkaar en controleren of deze matches.

```
const zijnGedraaideKaartenMatch = () => {
  let images = document.querySelectorAll("img");
  let srcs = []
  images.forEach(img => {
    let src = img.getAttribute("src")
    if(isKaartGedraaid(img)){
      srcs.push(src);
    }
  })
  if(srcs[0]===srcs[1]){
    return true;
  }else{
    return false;
  }
}
```

Om een paar te verwijderen, overlopen we alle afbeeldingen en controlen we opnieuw welke niet de achterkant zijn en zetten deze dan toch op de achterkant om geen problemen te hebben.

Via **visibility** kan je een element onzichtbaar doen worden en ervoor zorgen dat hij nog steeds de ruimte inneemt. We controleren ook of dit het einde van het spel is. (laatste kaarten waren of niet).

```
const verwijderPaar = () => {
  let images = document.querySelectorAll("img");
  images.forEach(img => {
    if(isKaartGedraaid(img)){
      img.setAttribute("src", "images/achterkant.png");
      img.style.visibility = "hidden";
      img.parentElement.style.border = "none";
    }
  })
  global.isBusy = false;

  let audio = new Audio('sounds/correct.mp3');
  audio.play();

  controleerEindeSpel();
}
```

Ook hier controleren welke kaarten zijn gedraaid en zetten de afbeelding terug op de achterkant en zetten de border terug. Via de parentElement kan je de parent zijn eigenschappen bereiken.

```
const draaiKaartenTerug = () => {
  let images = document.querySelectorAll("img");

  let audio = new Audio('sounds/Kaart_terugdraaien.mp3');
  audio.play();

  images.forEach(img => {
    if (isKaartGedraaid(img)) {
      img.setAttribute("src", "images/achterkant.png");
      img.parentElement.style.border = "2px solid black";
    }
  });
  global.isBusy = false;
}
```

Hier controleren of er nog een afbeelding zichtbaar is en niet hidden is bij visibility.

```
const controleerEindeSpel = () => {
  let images = document.querySelectorAll("img");
  let nogZichtbaar = false;

  images.forEach(img => {
    if (img.style.visibility !== "hidden") {
      nogZichtbaar = true;
    }
  });

  if (!nogZichtbaar) {
    let audio = new Audio('sounds/winning_sound.mp3');
    audio.play();
    alert("Proficiat, je hebt het spel gewonnen!");
  }
}
```

We willen nu ook nog extra maken zodat we ook naar 3,4,5,... gelijke kaarten moeten hebben in dit spel.

Hiervoor voegen we een globale variabele toe genaamd **AANTAL_GELIJKE_KAARTEN**.

Bij vulKaarten gaan we via een for het aantal kaarten aanpassen zodat het juist is voor het aantal gelijke kaarten mogelijk te maken en op het einde geen kaarten meer te hebben.

```
const vulKaarten = () => {
  let images = geefAfbeeldingen();
  let kaarten = [];

  images.forEach(img => {
    for (let i = 0; i < global.AANTAL_GELIJKE_KAARTEN; i++) {
      kaarten.push(img);
    }
  });

  kaarten.sort(() => Math.random() - 0.5);
  return kaarten;
};
```

In draaiKaartOm kijken we gewoon of het aantal gedraaide kaarten nu gelijk is aan de globale variabele.

Omdat er meerdere kaarten zijn kunnen we niet reeks controleren.

We starten bij index 1 en controleren 0 met 1 dan 1 met 2 enzovoort tot alles is gecontroleerd. Als het 1 keer niet matched dan returnen we false.

```
const zijnGedraaideKaartenMatch = () => {
  let images = document.querySelectorAll("img");
  let srcs = []
  images.forEach(img => {
    let src = img.getAttribute("src")
    if(isKaartGedraaid(img)){
      srcs.push(src);
    }
  })
  for(let i = 1; i < srcs.length; i++) {
    if(!(srcs[i-1]===srcs[i])){
      return false;
    }
  }
  return true;
}
```